# Xiaotong Sun

## MSE806_Assessment2_Project_Report

2604_ITS

### Document Details

Submission ID
trn:oid:::3618:156975723

Submission Date
Nov 3, 2026, 4:25 PM GMT+13

Download Date
Nov 3, 2026, 4:30 PM GMT+13

File Name
MSE806_Assessment2_Project_Report.docx

File Size
134.8 KB

14 Pages

3,165 Words

17,939 Characters

# 1% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

## Filtered from the Report

- Bibliography
- Quoted Text
- Small Matches (less than 8 words)

## Match Groups

**3** Not Cited or Quoted 1%
Matches with neither in-text citation nor quotation marks

**0** Missing Quotations 0%
Matches that are still very similar to source material

**0** Missing Citation 0%
Matches that have quotation marks, but no in-text citation

**0** Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

## Top Sources

1% Internet sources
0% Publications
0% Submitted works (Student Papers)

## Match Groups

- **3** Not Cited or Quoted 1%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

## Top Sources

1% Internet sources
0% Publications
0% Submitted works (Student Papers)

## Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1 Internet
**bp0313807.github.io** <1%

2 Internet
**inventivehq.com** <1%

3 Internet
**www.promptfoo.dev** <1%

# Secure Intelligent Traffic Congestion Prediction and Monitoring System

## Task B Project Report

| Course | MSE806 Intelligent Transportation Systems |
|---|---|
| **Assessment** | Assessment 2 Group Project |
| **Group members** | Xiaotong Sun 270876224 Jingbiao Geng 270686148 Han Jia 270902385 |
| **Project source** | Adapted from LuthandoCandlovu TrafficPrediction under the MIT License |

## Abstract

This project transforms an open-source repository of traffic-prediction algorithms into a replicable and secure ITS prototype. The system predicts the current average road speed 5 minutes into the future for one road segment in the city centre. It produces 17,280 5-minute records, produces time and lag features, trains a histogram-based gradient boosting regressor, exposes predictions via FastAPI and displays results on a dashboard in a browser. There were 3,456 validation records when the split was chronological (80/20). The mean absolute error of the model was 2.6083 km/h, RMS error of the model was 3.2803 km/h and R squared value of the model was 0.8247. Compared to a persistence baseline (persisting at the previous speed as the next speed) it had a lower MAE of 40.39 percent. The assessment version also includes API-key authentication, limits on input size, browser origins, rate limits, audit logs, security response headers and restricted browser origins. 5 automated tests were success. The results indicate that the prototype is effective in communicating useful short horizon estimates efficiently while synthetic data and a single road segment and in-memory security design restrict external validity and production readiness.

## Introduction

In the field of intelligent transportation systems, the short duration traffic prediction has applications for route guidance, congestion monitoring and traffic operational planning. The useful prediction is not enough, the forecast has to pass through a software service to another component or another user. The reliability is thus contingent upon the ability of the model itself but also on the communication channel to provide the sensor features as well as return a result. An unprotected predictive endpoint may be accessed more than once with invalid values or accessed by unauthorized clients. The assessment problem is to create a working prediction service and discuss the dependence of efficiency and security to enable trustworthy use of the ITS.

They chose the public TrafficPrediction repository as a starting point as it already posed the problem as an end-to-end traffic forecasting system. An initial synthetic data generator, model scripts, an example FastAPI application and a simple dashboard were provided as part of the source repository. The group did not consider this material as the original assessment. In the code and report the repository, author, URL and commit are identified. The assessment contribution replaces the data process, removes broken/disabled components, adds a portable model of the gradient-boosted, introduces chronological assessment, generates reproducible evidence, secures the API and adds tests. This way, the benefit of an open-source starting point in education is maintained while engineering decisions made by the group are subject to audit.

The goal of the project is to approximate the average speed over the next five-minute stretch along one synthetic road section. It also seeks to bring that estimate to light via a small-scale testable communication service. In order to be successful, there must be a repeatable data and training pipeline, a model that does better than a baseline, an API that will not accept

unauthorized/invalid requests, and documentation that will allow another group member to run the demonstration. The report analyzes these results and provides a rationale for why this current prototype is not suitable to become a real traffic-management application.

## Literature Review

Traditional statistical traffic prediction approaches have given way to approaches that learn the non-linear relationship among a large number of sensor data. Lv et al. (2015) showed that accurate and timely traffic-flow information is of significance for an ITS deployment and suggested that deep architectures can be used to model spatio-temporal correlations. Whilst their findings corroborate the broader utility of learned traffic properties, the present project does not seek to replicate their network or city-wide evidence. The smaller model of the tree is used for the assessment, as the intention is to provide a clear prototype that is replicable and doesn't require much computing power.

Gradient Boosting is a method that merges several weak decision trees and as the iterations progress, it will correct the errors made by the previous iterations. The implementation in scikit-learn that uses a histogram is meant to bin data as it comes in when training and is efficient in training data sets of a minimum size of approximately 10,000 samples. The property matches the generated records (17,280) and does not require an external OpenMP runtime on the assessment computer as does the chosen repository's XGBoost package. The change was to enhance the portability of the system but also to alter the original architecture. The report thus does not say that the final results were obtained from XGBoost but rather gradient boosting.

Evaluation design is important because future patterns can affect training when randomly shuffling a time series. This project maintains the time-order of observations and splits the first 80 percent for training, and the last 20 percent for validation. It reports MAE since the value is

directly interpretable in kilometres per hour, RMSE because it gives more weight to the large errors and R squared because it describes explained variation relative to a mean prediction. A persistence baseline is useful for this short time horizon, as the learned performance is only valuable if it is better than what was previously observed.

The implementation is also influenced by security guidance. According to the OWASP API Security Top 10 2023, a few of the most prevalent API risks are broken authentication and unrestricted resource usage. According to the NIST AI Risk Management Framework, risk management is a lifecycle process involving risk governance, risk context mapping, risk behavior measurement and risk management. The sources also indicate that model accuracy, access control, and monitoring and documented limitations should be part of the same system evaluation. The prototype is therefore deployed to test authentication, validation, response headers and rate limiting—not as a purely theoretical consideration, but as one that is vital to security.

## Methodology

The new generator produces 1 record at 5-minute intervals for 60 days (17,280 observations). Each record contains a time stamp, segment identifier, vehicle volume, precipitation, temperature and an incident indicator, as well as a weekday and an hour. On weekdays, speed is lowered during rush hour in the morning and evening. A set of separate reductions are applied for incidents and precipitation, and random noise prevents the determination of the target by mechanical means. A fixed random seed will make the data reproducible. These values are not real values. They don't define real world traffic behaviour, they just offer controlled test data.

Feature engineering is used to group records by their time stamp and to generate the speed lags for the previous 3 time periods and the rolling mean of the 3. Also input to the model are hour, day of week, temperature, precipitation, vehicle volume, and status of incidences. The speed column has been shifted, so that each row is not using any later speeds. The first three rows are deleted as the lag values are not available. The code then splits the data into 80% chronological and 20% non-chronological parts. It fits a HistGradientBoostingRegressor using a learning rate of 0.08, 180 boosting iterations, max. 31 leaf nodes and L2 regularisation of 0.1. State is always set to 42.

The evaluation script loads the previously said model and makes predictions for all 3,456 validation observations. It calculates MAE, RMSE and R squared for the model and for a persistence baseline. It also calculates the total prediction time per the number of validation rows to get the mean model inference time per row. The script writes the results to JSON and creates a chart to compare the MAE and displays the actual speed and the predicted speed for the first validation day. The report utilizes these values generated values instead of the estimated entries made by hand.

The software communication part is implemented with FastAPI. Pydantic schema will disallow any additional fields or force all numeric values into a reasonable range. The prediction route needs an X-API-Key header, and does a constant-time comparison with the configured key. Only requests made from the two local dashboard origins are allowed. A sliding window limiting will support 30 request per client per minute. Middleware is responsible for adding its own request ID, logging the path, the status, and the duration of the request, sending a message to the clients to not cache predictions and appending a protection header with the MIME-sniffing information. The health endpoint is public; it does not require a prediction to test the availability of the service.

Testing is done via a local test client from FastAPI and a static test model. Five tests ensure the public health endpoint, rejection of missing key, rejection of out-of-range hour, successful prediction with security headers and blocking after hitting a rate-limit threshold. The demonstration workflow creates data, trains the model, evaluates the model, runs tests, starts API and serves static dashboard. There are documented commands for each step so that the group can recreate the evidence.

## Results and Analysis

The model with gradient boosted has an MAE of 2.6083 km/h, an RMSE of 3.2803 km/h, and an R squared of 0.8247 for the chronological validation period. The persistence baseline produced an MAE of 4.3756 km/h, an RMSE of 5.9816 km/h and an R squared value of 0.4172. The results showed that the learned model minimized the MAE by 40.39 per cent. The improvement shows this combined time, demand, weather, incident and lag features were able to capture patterns that were not available from the most recent speed alone.

The RMSE is larger than the MAE for both methods as some observations contain larger errors than the average prediction. This is important when there are congestion events as an average error may mask some misses that are operationally critical. The R squared value indicates that the model accounts for a significant portion of the variance during the validation period generated from the traffic it was trained on, but does not demonstrate performance on actual traffic. The same process is used for training and validation so that the same assumptions are made for peaks, incidents and weather for both periods. The evaluation then provides evidence of correctness of implementation and relative performance within the simulated scenario.

When the evaluation script predicted the validation set in a batch, the mean model inference time was about 0.004 milliseconds per validation record. This time varies from run to run but is slightly different. This is not the network latency or latency to parse a network message as a JSON or API middleware. Even with that restriction the measurement indicates that computing the models is unlikely to be the major constraint in a classroom demonstration. The API provided the dashboard sample's five minute forecast (44.00 km/h) including the segment identifier and forecast horizon.

All of the automated tests were successful. The findings validate the effectiveness of the controls in action under the conditions tested. Not penetration testing or independent security certification. The in-memory limiter will affect only one process. If each process has its own counter, a distributed deployment would be possible which would avoid the planned global limit.

| Method | MAE km/h | RMSE km/h | R squared |
|---|---|---|---|
| Persistence baseline | 4.3756 | 5.9816 | 0.4172 |
| Gradient boosting | 2.6083 | 3.2803 | 0.8247 |

*Table 1. Chronological validation results generated by evaluate_models.py*

## Security Assessment

The principle assets are the model availability, the model files, the traffic inputs, the forecast integrity and the API key and audit evidence. These potential risks are likely to include unauthorized prediction requests, too many automated calls, incorrect values, manipulated incident inputs, lost keys, unencrypted communication and misinterpretation of forecasts. The prototype directly deals with a number of threats. Requests not including the shared secret are denied by API-key authentication. Extra fields rejected and range checks reduce malformed-input risk. Rate limiting is used to restrict repeated calls from a client. Restricted CORS makes it

so that normal browsers on different origins can't read the API. The IDs and logs make it easier to track and understand, and the no-store responses minimize unwanted caching.

These controls enhanced the original repository that accepted all origins and accepted an unrestricted dictionary. But, there are still some gaps in the design. A shared API key is not user-specific or for role-based access control. The transport encryption relies on a future reverse proxy as the local demonstration is based on HTTP. Logs are not stored in a protected central store. Restart does not reset the rate limiting, and does not coordinate multiple instances. There is no mechanism which verifies the authenticity of a sensor nor the ability to verify that an attacker modified model files. In order to use the production, TLS, managed secrets, per-client identity, a shared rate-limit service, signed sensor messages, model-artifact integrity checks, central monitoring and incident-response procedures were required.

Model risk has an impact on security. Plausible but false values in the allowed range could be supplied by attackers or faulty sensors, without triggering schema validation, thus causing incorrect forecasts. In addition, as road layout or travel behaviour and even sensor calibration change, a drift may also decrease the accuracy. A deployment should track input distributions and error by time and location, unusual requests and repeated corrections by operators. Human review should be required for high impact decisions and they should be uncertain and show age. The current dashboard highlights the synthetic data and the short horizon; however, they do not determine a confidence interval.

The assessment data has low privacy exposure because there are no person or vehicle identifiers in the data. Number plates, device identifiers or exact trajectories could be sent to a real system which would result in obligations to personal information. When identifiers are not required for the training of the model, they should be removed before they are used.Those identifiers should be removed prior to model training if they are not needed. The operator should

establish purpose, retention, access and deletion policies and determine if route recommendations lead to a shift of noise or congestion to specific communities. These actions help to protect the system while upholding good ITS practice.

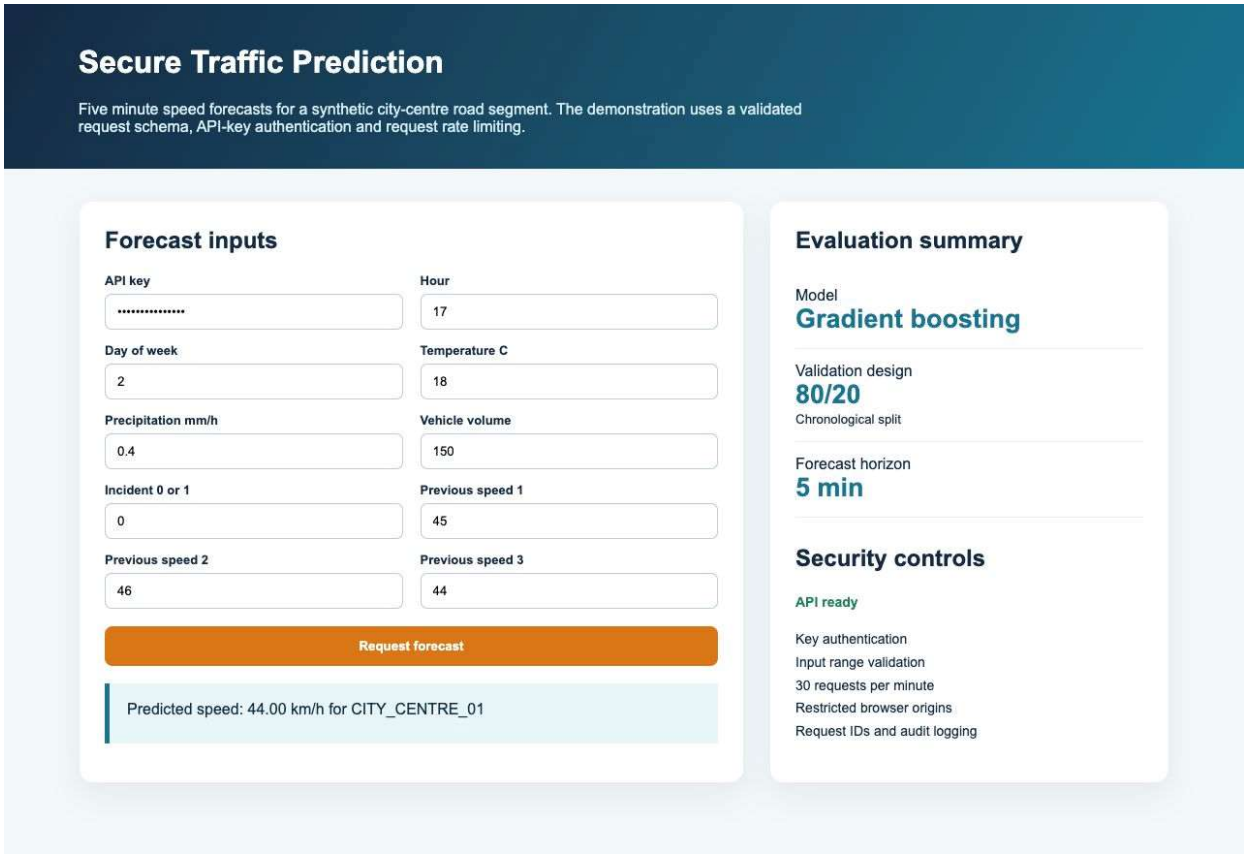



*Figure 1. Dashboard demonstration with an authenticated forecast*

## Discussion

The project is able to achieve the primary engineering goals within the scope of the prototype. It generates reproducible data, builds a working model, enhances a meaningful baseline, shares predictions via an API, and shows secure behaviour by conducting tests. The architecture also enforces separation of responsibilities. The data can change without having to rewrite the dashboard, and there is one place for validation and log purposes: API controls. The reproducible JSON results allow for the ability to attribute report claims to code output.

The project also demonstrates a balance between the technical aspirations and the practical reliability. The checker out repository selected contained both XGBoost and an LSTM, but was not checked out and did not contain any data files, models, or dependences and failed to provide a working portable environment. The first XGBoost wheel required an OpenMP library which wasn't installed on the assessment computer. The group decided to use the histogram gradient boosting, implemented in the package scikit-learn, in order to keep the final workflow system independent and to test it without requiring an additional system dependency. This choice is to increase the breadth of the architecture while improving the reproducibility. The source repository is still credited since it provided the initial idea and coding structure.

The most significant one is the external validity. The generator encodes the relationships that the model is then able to learn, and one segment is unable to represent network interactions, events, signal timing and spatial spillover. Further research should involve using authorized sensor information from several road sections, ensure a final unseen testing period and compare performance in normal, peak, incident and severe weather conditions. The group should also consider the impact on fairness of any route recommendation, along with calibration and prediction intervals. Load tests, failure recovery, encrypted communications and monitoring of multiple service instances would be required for a production pilot.

Important, an operational evaluation should also specify what constitutes a useful forecast for the planned decision. For a general congestion display, a two or three kilometre per hour error could be acceptable; however, for signal control and directing emergency vehicles to alternate routes, such error is not likely to be tolerable. The road authority should establish thresholds for warning, fallback and human intervention prior to the start of a pilot. It should compare the learned model with simpler seasonal and statistical baselines, track end-to-end response times under realistic loads and record service availability during model updates. The

interface should show up when the sensor inputs are stale, and should default to the last sensor observation that was verified when the model or feature service is not available. These requirements relate model metrics to reliable software behavior. They also help to avoid believing that a high average R squared is healthy or helpful for all the individual forecasts.

## Conclusion and Self Reflection

The completed prototype shows the intelligent transportation model as a SIC (Software Information Communication) system that is testable. The gradient-boosted model obtained 2.6083 km/h in MAE and reduced it by 40.39 percent compared to the persistence baseline during the synthetic validation period. Access control, validation, rate limiting and traceability have been integrated with prediction instead of just included as written recommendations, as indicated by the secured API, dashboard and five passing tests.

In addition to the choice of an algorithm, Xiaotong Sun learned the value of evaluating time-series and the importance of reproducibility. Jingbiao Geng found that a working endpoint is not guaranteed without authentication, input constraints and monitoring. Han Jia discovered that evidence that can be regenerated and a clear separation between measured results and limitations are crucial to a good presentation. The group also got to learn about adapting open source work transparently. The final system keeps all the attributions and reports a real replacement of XGBoost and which contributions are in the group. Validation with authorized real data and a more robust production security architecture, not the deployment of the classroom prototype, is the next responsible step.

# Group Contributions

| Member and role | Implementation contribution | Presentation |
|---|---|---|
| Xiaotong Sun 270876224 Data and Model Engineer | Generator, features, model and reproducibility | Part 1 |
| Jingbiao Geng 270686148 API and Security Engineer | API controls, logging and tests | Part 2 |
| Han Jia 270902385 Evaluation and Communication Lead | Metrics, dashboard, report and integration | Part 3 |

## References

Candlovu, L. (2025). TrafficPrediction [Computer software]. GitHub. https://github.com/LuthandoCandlovu/TrafficPrediction

Lv, Y., Duan, Y., Kang, W., Li, Z., & Wang, F. Y. (2015). Traffic flow prediction with big data: A deep learning approach. IEEE Transactions on Intelligent Transportation Systems, 16(2), 865–873. https://doi.org/10.1109/TITS.2014.2345663

National Institute of Standards and Technology. (2023). Artificial Intelligence Risk Management Framework AI RMF 1.0. https://doi.org/10.6028/NIST.AI.100-1

OWASP Foundation. (2023). OWASP API Security Top 10 2023. https://owasp.org/API-Security/

Scikit-learn developers. (2026). HistGradientBoostingRegressor. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.HistGradientBoostingRegressor.html